

MODULE 1

SYLLABUS

Basic Concepts of Data Structures Definitions; Data Abstraction; Performance Analysis - Time & Space Complexity, Asymptotic Notations; Polynomial representation using Arrays, Sparse matrix (Tuple representation); Stacks and Queues - Stacks, Multi-Stacks, Queues, Circular Queues, Double Ended Queues; Evaluation of Expressions- Infix to Postfix, Evaluating Postfix Expressions.

DATA STRUCTURES AND DATA ABSTRACTION

A data structure is a way to organize, manage, and store data so that it can be used efficiently. It defines how data is arranged in memory and how operations like insertion, deletion, searching, and updating can be performed. In simple words: Data structures are like containers that hold data together in a certain format so that computers can easily access and modify them. More formally "A data structure is a mathematical and logical model of a particular organization of data. It describes the relationship among the data elements, and the operations that can be performed on them."

Key Points about Data Structures:

- They help in organizing large amounts of data easily.
- They improve the efficiency of algorithms.
- They decide how fast and efficient programs work.
- Different problems need different data structures for best performance.

Types of Data Structures:

Type	Examples
Linear Data Structures	Array, Linked List, Stack, Queue
Non-linear Data Structures	Tree, Graph
Hash-based Structures	Hash Table, Hash Map

Real-life Examples:

- Array: Like a row of boxes storing numbers: [5, 10, 15, 20]
- Stack: Like a pile of plates (Last-In-First-Out).
- Queue: Like a line of people waiting (First-In-First-Out).
- Tree: Like a family tree showing relationships.
- Graph: Like a map showing cities and roads.

Data Abstraction

Data abstraction in data structures means hiding the low-level details of how data is stored and focusing only on what operations can be performed on the data. It separates what a data structure does from how it does it.

- Abstraction = Hiding complexity, showing only the necessary operations.
- Like driving a car — you don't need to know how the engine works to drive.
Similarly, when you use a Stack, you only care about push and pop, not how the stack is maintained internally.

Data Abstraction refers to the idea of defining a data structure by the operations that can be performed on it and the properties these operations must satisfy, without specifying how these operations are implemented.

Key points:

- Focus on what the data can do, not how it does it.
- Helps in creating Abstract Data Types (ADTs).
- Implementation details are hidden from the user

Concepts in Data Abstraction:

- Abstract Data Types (ADT):
Data abstraction leads to the concept of an ADT, where only essential operations are exposed.

- Interface vs Implementation:
 - Interface: What you can do (e.g., Insert, Delete, Search)
 - Implementation: How it is done (e.g., using an array or linked list)

Example

Stack ADT

Logical View (Abstraction):

- A *stack* is a collection of elements where:
 - Insertion (push) and deletion (pop) happen at one end only (top).
 - LIFO (Last In First Out) behavior.

Implementation (Data Structure):

- Array (fixed size stack).
- Linked list (dynamic size stack).

Example:

- Logical operations: push(x), pop(), peek().
- Implementation:
 - Use an array where top points to the last inserted element.
 - Or use a linked list where insertion and deletion are done at the head

Advantages of Data Abstraction in Data Structures:

- Makes programming easier and error-free.
- Allows you to change the internal implementation without affecting users.
- Encourages modularity and code reuse.

PERFORMANCE ANALYSIS

Performance analysis means studying how efficient an algorithm or data structure is. Mainly, we check two things:

- Time Complexity — How much time (number of operations) the algorithm takes.
- Space Complexity — How much memory (space) the algorithm uses.

Goal: To predict how the algorithm will behave as the input size grows without running the code.

TIME COMPLEXITY

- Measures the **amount of time** an algorithm takes to complete.
- It is expressed as a function of the **input size n** .
- We usually use **Big-O notation** (e.g., $O(n)$, $O(\log n)$, $O(n^2)$) to express it.

Why it matters?

Because we want algorithms that are **fast even for large inputs**.

Example for Time Complexity:

Suppose we have a simple loop:

```
for (int i = 0; i < n; i++) {
    printf("%d ", i);
}
```

- The loop runs **n times**.
- Each printf is a constant-time operation.
- So, **Time Complexity = $O(n)$** (linear time).

Types of Common Time Complexities:

Complexity	Name	Example
$O(1)$	Constant time	Accessing array element
$O(\log n)$	Logarithmic time	Binary search
$O(n)$	Linear time	Traversing an array
$O(n \log n)$	Linearithmic time	Merge sort, Heap sort

$O(n^2)$	Quadratic time	Bubble sort, Selection sort
$O(2^n)$	Exponential time	Solving Tower of Hanoi

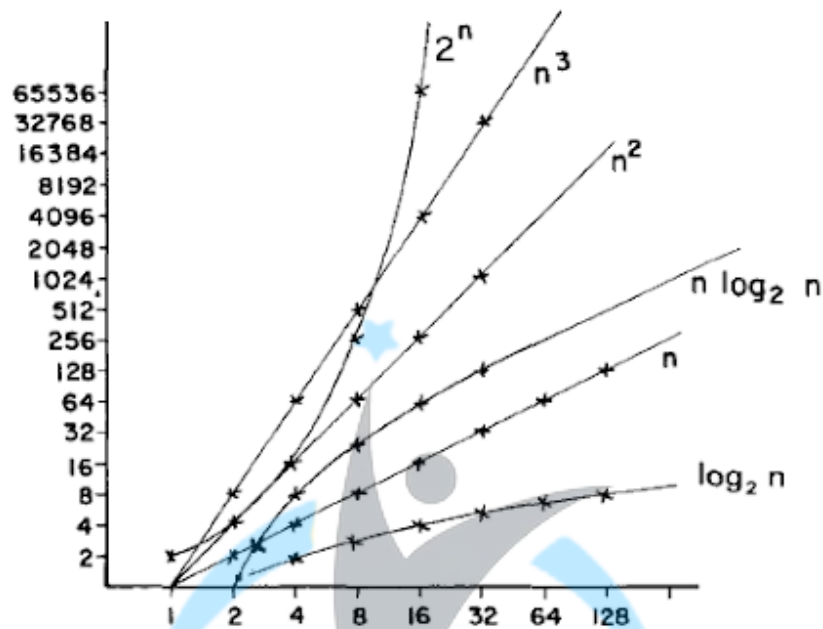


Figure 1.6: Rate of Growth of Common Computing Time Functions

$\log_2 n$	n	$n \log_2 n$	n^2	n^3	2^n
0	1	0	1	1	2
1	2	2	4	8	4
2	4	8	16	64	16
3	8	24	64	512	256
4	16	64	256	4096	65536
5	32	160	1024	32768	2, 147, 483, 648

Figure 1.7: Values for Computing Functions

SPACE COMPLEXITY

- Measures the amount of memory an algorithm needs to complete its execution.
- Memory includes:
 - Variables
 - Data structures (arrays, linked lists)
 - Recursion stack space

Why it matters?

Because limited memory can slow down or crash a program.

Example for Space Complexity:

Suppose:

```
int a, b, c; // 3 integers
```

- These use constant memory $\rightarrow$ Space Complexity = $O(1)$.

Now:

```
int arr[n]; // array of n elements
```

- Memory used grows with $n \rightarrow$ Space Complexity = $O(n)$.

Example Combining Both:

Suppose we want to find the sum of all elements in an array:

```
int sum(int arr[], int n) {  
    int total = 0;  
    for (int i = 0; i < n; i++) {  
        total += arr[i];  
    }  
    return total;  
}
```

- Time Complexity:
The loop runs n times $\rightarrow O(n)$.
- Space Complexity:
Only total variable used (ignoring input array) $\rightarrow O(1)$.

Why Performance Analysis is Important?

- Helps to choose the best algorithm.
- Helps to predict performance for large inputs.
- Avoids wasting time, memory, and power.

ASYMPTOTIC ANALYSIS

Asymptotic Analysis is the process of analyzing the efficiency (especially time and space complexity) of an algorithm as the input size (n) grows very large.

- It focuses on the behavior of the algorithm for large inputs.
- It ignores smaller constants and less significant terms.
- It helps to compare algorithms independently of machine speed, compiler optimizations, or programming languages.

Simply: It answers "How does the running time (or space) grow as $n \rightarrow \infty$?"

Why do we need Asymptotic Analysis?

- To evaluate and compare the scalability of algorithms.
- To predict performance when input size becomes very large.
- To find better algorithms for real-world, large-scale problems.

Key Focus of Asymptotic Analysis:

- Concentrates only on the dominant term (highest degree).
- Constants (like 5, 1000) are ignored.
- Lower-degree terms (like n , $\log n$) are ignored if a higher degree exists.

What are Asymptotic Notations?

- Asymptotic notations are mathematical tools to describe the running time or space requirements of an algorithm in terms of input size (n).
- They help us understand how an algorithm behaves when n becomes very large (i.e., in the "asymptotic" case).
- Focus: Only the dominant factors are considered, ignoring smaller terms and constant coefficients.

Example:

If time taken = $3n^2 + 5n + 20$,
when n is very large, the term $3n^2$ dominates.
Thus, time complexity = $O(n^2)$.

Why Asymptotic Notations?

- To compare algorithms independently of hardware or programming language.
- To predict performance for very large inputs.

Formal Definition:

Asymptotic analysis uses mathematical functions to describe the growth rate of an algorithm with respect to input size n , using notations like:

- Big O (O) — Worst case
- Big Omega (Ω) — Best case
- Big Theta (Θ) — Average/Tight case

Types of Asymptotic Notations

Notation	Name	Purpose
$O()$	Big O	Upper bound (worst case)
$\Omega()$	Big Omega	Lower bound (best case)
$\Theta()$	Big Theta	Tight bound (average case)

1. Big O Notation (O) — Upper Bound

- Describes the maximum time or maximum number of steps.
- "At most this much time is needed."
- Used for worst-case analysis.

Formal Definition:

A function $f(n) = O(g(n))$ if there exist positive constants c and n_0 such that for all $n \geq n_0$,
 $f(n) \leq c \times g(n)$

Example:

Bubble Sort has $O(n^2)$ in the worst case.

If $f(n) = 3n^2 + 4n + 2$,

then $f(n) = O(n^2)$ because as n grows, n^2 dominates.

2. Big Omega Notation (Ω) — Lower Bound

- Describes the minimum time an algorithm can take.
- "At least this much time is needed."
- Used for best-case analysis.

Formal Definition:

A function $f(n) = \Omega(g(n))$ if there exist positive constants c and n_0 such that for all $n \geq n_0$,
 $f(n) \geq c \times g(n)$

Example:

In Bubble Sort, if the array is already sorted,

Best-case time = $\Omega(n)$ (only one pass needed).

3. Big Theta Notation (Θ) — Tight Bound

- Describes an exact bound.
- "Both upper and lower bounds match."
- Used for average case or tight bound analysis.

Formal Definition:

A function $f(n) = \Theta(g(n))$ if there exist positive constants c_1 , c_2 , and n_0 such that for all $n \geq n_0$,
 $c_1 \times g(n) \leq f(n) \leq c_2 \times g(n)$

Example:

If $f(n) = 5n + 20$, then

$f(n) = \Theta(n)$ because the function grows linearly with n .

Quick Example Summary

Algorithm	Best Case	Average Case	Worst Case
Linear Search	$\Omega(1)$	$\Theta(n)$	$O(n)$
Binary Search	$\Omega(1)$	$\Theta(\log n)$	$O(\log n)$
Bubble Sort	$\Omega(n)$	$\Theta(n^2)$	$O(n^2)$

Notation	Meaning	Describes
$O(g(n))$	$\leq c \times g(n)$ (upper bound)	Worst case
$\Omega(g(n))$	$\geq c \times g(n)$ (lower bound)	Best case
$\Theta(g(n))$	Between two multiples of $g(n)$	Exact/Tight

Easy way to remember:

- O — Overestimate (maximum steps)
- Ω — Optimistic (minimum steps)
- Θ — Tight (exact steps)

Choosing the Best Algorithm

- When engineers design software, many algorithms are available to solve a problem.
- Asymptotic analysis helps compare these algorithms based on time and space efficiency.
- For small inputs, any algorithm may work, but for large inputs, only efficient algorithms can save time and resources.

Example:

- Linear search: $O(n)$
- Binary search: $O(\log n)$
(When n is large, binary search is *much* faster.)

In Short:

Asymptotic Analysis Helps In	How it Helps
Choosing algorithms	Pick the fastest one for large data
Planning for scaling	Make sure system works even if data grows 1000×
Saving resources	Save memory, processor time
Building real-time systems	Ensure guaranteed response times
Optimizing programs	Focus on critical slow parts

Asymptotic Analysis is a powerful mathematical tool that helps computer engineers to:

- Design efficient algorithms
- Predict performance
- Ensure scalability and reliability
- Build faster, smarter, and safer software systems

POLYNOMIAL ADDITION USING ARRAYS

We can represent a polynomial as a collection of terms and every term is having a coefficient and exponent. So, we can represent this as a list of terms having coefficients and exponents.

Each term of a polynomial (like $3x^2$, $5x^1$, $-7x^0$) can be stored in a **structure** having two parts:

- **coefficient** (like 3, 5, -7)
- **exponent** (like 2, 1, 0)

An **array of such structures** represents the whole polynomial.

Structure Definition (Example in C)

```
struct Term {  
    int coeff; // Coefficient of the term  
    int expo;  // Exponent of the term  
};
```

And then you can declare:

```
struct Term poly1[10], poly2[10], polyResult[20];  
to represent polynomials.
```

Algorithm to Add Two Polynomials

Assumption:

Both polynomials are sorted in **decreasing order of exponents**.

Steps:

1. Initialize three pointers (indexes):
 i = 0 for first polynomial (poly1),
 j = 0 for second polynomial (poly2),
 k = 0 for result polynomial (polyResult).
2. Repeat while both *i* and *j* are within their respective array sizes:
 - If $\text{poly1}[i].\text{expo} == \text{poly2}[j].\text{expo}$:
 - Add their coefficients.
 - Store the sum and the common exponent in $\text{polyResult}[k]$.
 - Increment *i*, *j*, and *k*.
 - Else if $\text{poly1}[i].\text{expo} > \text{poly2}[j].\text{expo}$:
 - Copy $\text{poly1}[i]$ to $\text{polyResult}[k]$.
 - Increment *i* and *k*.
 - Else:
 - Copy $\text{poly2}[j]$ to $\text{polyResult}[k]$.
 - Increment *j* and *k*.
3. Copy any remaining terms from poly1 (if any) to polyResult.
4. Copy any remaining terms from poly2 (if any) to polyResult.
5. Done.

Example

Suppose:

Polynomial 1: $5x^3 + 4x^2 + 2x^0$

Polynomial 2: $5x^3 + 5x^1 + 5x^0$

Represented as Arrays:

poly1	poly2
(5, 3)	(5, 2)
(4, 2)	(5, 1)
(2, 0)	(5, 0)

Step-by-step Addition:

Step	Compare	Action	Result Term
1	3 vs 2	$3 > 2 \rightarrow$ copy (5,3) from poly1	(5,3)

2	2 vs 2	Equal → add 4+5=9, exponent=2	(9,2)
3	0 vs 1	0 < 1 → copy (5,1) from poly2	(5,1)
4	0 vs 0	Equal → add 2+5=7, exponent=0	(7,0)

Final polyResult:

coeff	expo
5	3
9	2
5	1
7	0

Resulting Polynomial: $5x^3+9x^2+5x^1+7$

C Program- Polynomial Addition using arrays

```
#include <stdio.h>
```

```
struct Term {
    int coeff;
    int expo;
};
```

```
void addPolynomials(struct Term poly1[], int n1, struct Term poly2[], int n2, struct Term polyResult[], int *nResult) {
    int i = 0, j = 0, k = 0;
```

```
    while (i < n1 && j < n2) {
        if (poly1[i].expo == poly2[j].expo) {
            polyResult[k].coeff = poly1[i].coeff + poly2[j].coeff;
            polyResult[k].expo = poly1[i].expo;
            i++;
            j++;
            k++;
        }
```

```
        else if (poly1[i].expo > poly2[j].expo) {
            polyResult[k] = poly1[i];
            i++;
            k++;
        }
```

```
        else {
            polyResult[k] = poly2[j];
            j++;
            k++;
        }
    }
```

```
    // Copy remaining terms of poly1
    while (i < n1) {
        polyResult[k] = poly1[i];
        i++;
        k++;
    }
```

STUDENT ASSIST CELL
an initiative of KSU CET unit

```

// Copy remaining terms of poly2
while (j < n2) {
    polyResult[k] = poly2[j];
    j++;
    k++;
}

*nResult = k; // Total number of terms in result
}

void displayPolynomial(struct Term poly[], int n) {
    for (int i = 0; i < n; i++) {
        printf("%dx^%d", poly[i].coeff, poly[i].expo);
        if (i != n - 1) {
            printf(" + ");
        }
    }
    printf("\n");
}

int main() {
    struct Term poly1[] = { {5, 3}, {4, 2}, {2, 0} };
    struct Term poly2[] = { {5, 2}, {5, 1}, {5, 0} };
    struct Term polyResult[20];
    int nResult;

    int n1 = 3; // Number of terms in poly1
    int n2 = 3; // Number of terms in poly2

    printf("First Polynomial: ");
    displayPolynomial(poly1, n1);

    printf("Second Polynomial: ");
    displayPolynomial(poly2, n2);

    addPolynomials(poly1, n1, poly2, n2, polyResult, &nResult);

    printf("Sum of Polynomials: ");
    displayPolynomial(polyResult, nResult);

    return 0;
}

```

Output

First Polynomial: $5x^3 + 4x^2 + 2x^0$
 Second Polynomial: $5x^2 + 5x^1 + 5x^0$
 Sum of Polynomials: $5x^3 + 9x^2 + 5x^1 + 7x^0$

SPARSE MATRIX

A sparse matrix is a special type of matrix in which most of the elements are zero or have some default value. In contrast, a dense matrix is one in which most of the elements are non-zero. Sparse matrices are used in various fields, including computer science, numerical analysis, and scientific computing, when dealing with large datasets or systems where efficiency and memory usage are crucial. As a rule of thumb, if **2/3 of the total elements in a matrix are zeros, it can be called a sparse matrix**. Using a sparse matrix representation — where only the non-zero values are stored — the space used for representing data and the time for scanning the matrix are reduced significantly. The sparsity of a matrix refers to the proportion of its elements that are zero or some other default value (i.e., the number of zero elements divided by the total number of elements in the matrix). It is often expressed as a percentage or a fraction. To compute the sparsity of a matrix, you can follow these steps:

Count the number of zero elements in the matrix.

Determine the total number of elements in the matrix.

Calculate the sparsity using the formula:

$$\text{Sparsity (\%)} = (\text{Number of Zero Elements} / \text{Total Number of Elements}) * 100$$

Note that most of the matrix elements will be zero since only non-zero values are important for data analysis. In addition to a large amount of space used, there will be a computational time problem because all elements will be scanned to access non-zero elements. This process yields a computational complexity problem.

To overcome these problems, we can use different data structures to represent a sparse matrix. the following is a 3 tuple representation. **The first row of sparse matrix always specifies the number of rows, number of columns and number of non zero elements in the matrix**

Rows	Columns	Values
5	6	6
0	4	9
1	1	8
2	0	4
2	2	2
3	5	5
4	2	2

TRANSPOSE OF A SPARSE MATRIX

Transpose of a matrix means **rows become columns and columns become rows**. In sparse matrix representation, **for each term, swap row and column**. Example:

Consider a 3x3 matrix

0 5 0
3 0 0
0 0 6

Sparse Representation (with Metadata):

(First row is **metadata**: rows, columns, total non-zero elements.)

Index	Row	Column	Value
0	3	3	3
1	0	1	5
2	1	0	3
3	2	2	6

Now let's find Transpose

Transpose means swapping the row and column for each element.

Metadata for transpose:

- Rows become Columns
- Columns become Rows

So metadata: (3, 3, 3)

So the transposed sparse matrix (with metadata):

Index	Row	Column	Value
0	3	3	3

1	0	1	3
2	1	0	5
3	2	2	6

And if you reconstruct the full matrix from this sparse data, it will look like:

```

0    3    0
5    0    0
0    0    6

```

Algorithm: Transpose of Sparse Matrix

Input:

- A sparse matrix `sparse[]` in 3-tuple form where:
 - `sparse[0]` stores (number of rows, number of columns, number of non-zero elements).
 - `sparse[1..n]` store (row, column, value) of non-zero elements.

Output:

- `transpose[]` array representing the transposed matrix.

Steps:

1. **Read** the `sparse[]` array, including metadata and non-zero elements.
2. **Initialize** the first element (`transpose[0]`) of transposed matrix:
 - `transpose[0].row = sparse[0].col`
 - `transpose[0].col = sparse[0].row`
 - `transpose[0].value = sparse[0].value`
3. **For each column i (0 to number of columns - 1)** of the original matrix:
 - **For each non-zero element** (from index 1 to n):
 - **If** the column index of the element is i:
 - Create a new entry in `transpose[]`:
 - `transpose[k].row = sparse[j].col`
 - `transpose[k].col = sparse[j].row`
 - `transpose[k].value = sparse[j].value`
 - Increment k.
4. **End**

Implementation of Transpose in C

```
#include <stdio.h>
struct Term {
    int row;
    int col;
    int value;
};

void transpose(struct Term sparse[], struct Term transpose[]) {
    int i, j, n;
    n = sparse[0].value; // number of non-zero elements
    // Step 1: Metadata for transpose
    transpose[0].row = sparse[0].col;
    transpose[0].col = sparse[0].row;
    transpose[0].value = sparse[0].value;
    // Step 2: Swap row and col for each non-zero element
    int k = 1; // index for transpose array
    for (i = 0; i < sparse[0].col; i++) { // for each column of original
        for (j = 1; j <= n; j++) { // check all elements
            if (sparse[j].col == i) {
                transpose[k].row = sparse[j].col;
                transpose[k].col = sparse[j].row;
                transpose[k].value = sparse[j].value;
                k++;
            }
        }
    }
}

void display(struct Term sparse[]) {
    int n = sparse[0].value; // number of non-zero elements
    printf("Row Col Value\n");
    for (int i = 0; i <= n; i++) {
        printf("%3d %4d %5d\n", sparse[i].row, sparse[i].col, sparse[i].value);
    }
}

int main() {
    int rows, cols, n;
    printf("Enter number of rows, columns, and non-zero elements: ");
    scanf("%d %d %d", &rows, &cols, &n);
    struct Term sparse[50], transposed[50];
    sparse[0].row = rows;
    sparse[0].col = cols;
    sparse[0].value = n;
    printf("Enter the non-zero elements (row column value):\n");
    for (int i = 1; i <= n; i++) {
        scanf("%d %d %d", &sparse[i].row, &sparse[i].col, &sparse[i].value);
    }
}
```

```

printf("\nOriginal Sparse Matrix Representation:\n");
display(sparse);
transpose(sparse, transposed);
printf("\nTransposed Sparse Matrix Representation:\n");
display(transposed);
return 0;
}

```

Output

Enter number of rows, columns, and non-zero elements: 3 3 3

Enter the non-zero elements (row column value):

0 1 5

1 0 3

2 2 6

Original Sparse Matrix Representation:

Row Col Value

3 3 3

0 1 5

1 0 3

2 2 6

Transposed Sparse Matrix Representation:

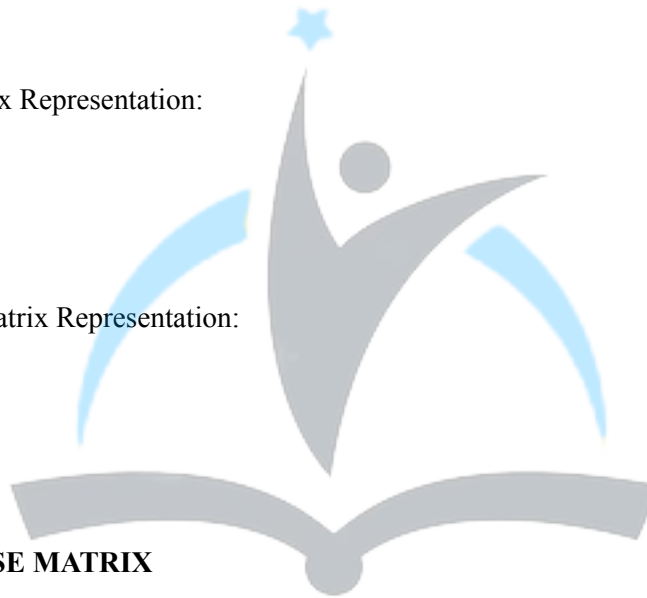
Row Col Value

3 3 3

0 1 3

1 0 5

2 2 6



ADDITION OF SPARSE MATRIX

Lets consider two sparse matrix representation A and B

A:

Index	Row	Column	Value
0	3	3	3
1	0	0	5
2	0	2	7
3	2	1	6

STUDENT ASSIST CELL
an initiative of KSU CET unit

B:

Index	Row	Column	Value
0	3	3	2
1	0	0	3
2	2	2	8

Perform Addition

- Compare A[1] and B[1]:
 - Same position (0,0). Add values: $5 + 3 = 8 \rightarrow (0,0,8)$
- Compare A[2] and B[2]:
 - A[2] (0,2) comes before B[2] (2,2). Copy A[2] $\rightarrow (0,2,7)$
- Now A[3] and B[2]:
 - A[3] (2,1) comes before B[2] (2,2). Copy A[3] $\rightarrow (2,1,6)$
- No more A. Now copy remaining B[2] $\rightarrow (2,2,8)$

Resultant Sparse Matrix C

Index	Row	Column	Value
0	3	3	4
1	0	0	8
2	0	2	7
3	2	1	6
4	2	2	8

Algorithm: Addition of Two Sparse Matrices

Input:

- Two sparse matrices A[] and B[] (both in 3-tuple form with metadata in the first row).

- Each row of A[] and B[] has (row, column, value).

Output:

- A sparse matrix C[] representing A + B.

Steps:

1. **Check if the matrices have the same dimensions:**
 - If A[0].row != B[0].row or A[0].col != B[0].col, then **Addition is not possible**. Stop.
2. **Initialize pointers:**
 - i = 1 (for A[])
 - j = 1 (for B[])
 - k = 1 (for C[])
3. **Set C[0] metadata:**
 - C[0].row = A[0].row
 - C[0].col = A[0].col
 - C[0].value = 0 (for now; update later)
4. **Compare and Merge:**
 - While i <= A[0].value and j <= B[0].value, do:
 - If (A[i].row < B[j].row) or (A[i].row == B[j].row and A[i].col < B[j].col):
 - Copy A[i] to C[k]
 - Increment i and k
 - Else if (B[j].row < A[i].row) or (B[j].row == A[i].row and B[j].col < A[i].col):
 - Copy B[j] to C[k]
 - Increment j and k
 - Else (Same position):
 - Add the values: sum = A[i].value + B[j].value
 - If sum != 0:
 - C[k].row = A[i].row
 - C[k].col = A[i].col
 - C[k].value = sum
 - Increment k
 - Increment both i and j
5. **Copy Remaining Elements:**
 - While i <= A[0].value:
 - Copy A[i] to C[k]
 - Increment i and k
 - While j <= B[0].value:
 - Copy B[j] to C[k]
 - Increment j and k
6. **Update Metadata:**

- Set $C[0].value = k - 1$

7. End

C Program for Sparse Matrix Addition

```
#include <stdio.h>
#define MAX 100
// Define structure for Sparse Matrix elements
typedef struct {
    int row;
    int col;
    int value;
} Element;
int main() {
    Element A[MAX], B[MAX], C[MAX];
    int i, j, k;
    int m, n, tA, tB; // rows, columns, non-zero elements in A and B
    // Input Sparse Matrix A
    printf("Enter rows, columns and number of non-zero elements of Matrix A: ");
    scanf("%d %d %d", &A[0].row, &A[0].col, &A[0].value);
    tA = A[0].value;
    printf("Enter elements (row column value) for Matrix A:\n");
    for(i = 1; i <= tA; i++) {
        scanf("%d %d %d", &A[i].row, &A[i].col, &A[i].value);
    }
    // Input Sparse Matrix B
    printf("Enter rows, columns and number of non-zero elements of Matrix B: ");
    scanf("%d %d %d", &B[0].row, &B[0].col, &B[0].value);
    tB = B[0].value;
    printf("Enter elements (row column value) for Matrix B:\n");
    for(i = 1; i <= tB; i++) {
        scanf("%d %d %d", &B[i].row, &B[i].col, &B[i].value);
    }
    // Check if addition possible
    if(A[0].row != B[0].row || A[0].col != B[0].col) {
        printf("Addition not possible. Matrix dimensions do not match.\n");
        return 0;
    }
    i = j = k = 1; // Initialize pointers
    C[0].row = A[0].row;
    C[0].col = A[0].col;
    // Merge and Add
    while(i <= tA && j <= tB) {
        if(A[i].row < B[j].row || (A[i].row == B[j].row && A[i].col < B[j].col)) {
            C[k++] = A[i++];
        }
        else if(B[j].row < A[i].row || (B[j].row == A[i].row && B[j].col < A[i].col)) {
```

```

        C[k++] = B[j++];
    }
    else { // Same position
        int sum = A[i].value + B[j].value;
        if(sum != 0) {
            C[k].row = A[i].row;
            C[k].col = A[i].col;
            C[k].value = sum;
            k++;
        }
        i++;
        j++;
    }
}
// Copy remaining elements of A
while(i <= tA) {
    C[k++] = A[i++];
}
// Copy remaining elements of B
while(j <= tB) {
    C[k++] = B[j++];
}
C[0].value = k - 1; // Update number of non-zero elements
// Display Result
printf("\nResultant Sparse Matrix (row column value):\n");
for(i = 0; i < k; i++) {
    printf("%d %d %d\n", C[i].row, C[i].col, C[i].value);
}
return 0;
}

```

Output

Enter rows, columns and number of non-zero elements of Matrix A: 3 3 3

Enter elements (row column value) for Matrix A:

0 0 5

0 2 7

2 1 6

Enter rows, columns and number of non-zero elements of Matrix B: 3 3 2

Enter elements (row column value) for Matrix B:

0 0 3

2 2 8

Resultant Sparse Matrix (row column value):

3 3 4

0 0 8

0 2 7

2 1 6

2 2 8

STUDENT ASSIST CELL

an initiative of KJ-SO GET unit

STACK USING ARRAYS

A stack is a linear data structure that follows the Last In First Out (LIFO) principle.

- Last In First Out (LIFO) means:
→ The element added last is the first one to be removed.

Think of a stack of plates:

- You add (push) plates on top.
- You remove (pop) plates from the top.

Basic Stack Operations:

Operation	Meaning
push(x)	Add element x to the top.
pop()	Remove the topmost element.
peek()	View the topmost element (without removing).
isEmpty()	Check if the stack is empty.

How to Implement a Stack Using Arrays?

We maintain:

- An array to hold the elements
- An integer top that keeps track of the index of the top element
 - If stack is empty, top = -1
 - When we push, we increment top
 - When we pop, we decrement top

Important Points:

- Stack Overflow happens when you push into a full stack.
- Stack Underflow happens when you pop from an empty stack.
- This stack implementation has a fixed size (MAX), so it is static.
- You can also implement a dynamic stack using linked lists

Algorithms for Stack Operations

We assume:

- stack[MAX] is an array to store elements.
- top is an integer that tracks the top of the stack.
- Initially, top = -1.

Algorithm for PUSH (Insert an element)

1. If $top == MAX - 1$ then
 Output "Stack Overflow"
 Exit
2. Else
 $top = top + 1$
 $stack[top] = item$
3. End If
4. Return

Algorithm for POP(Remove top element)

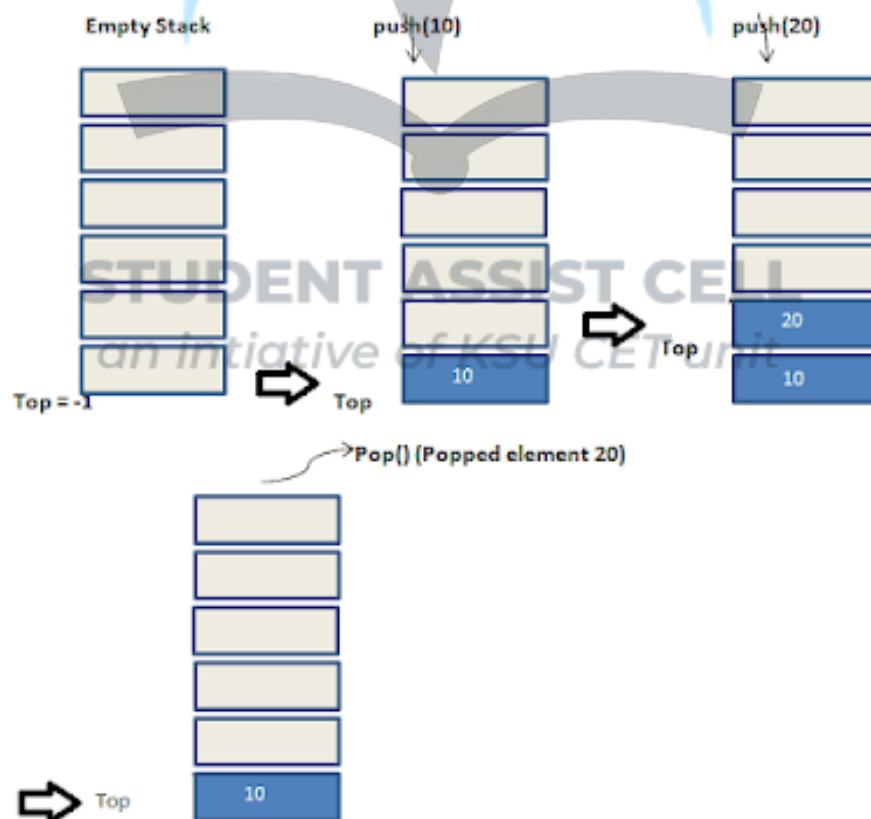
1. If $\text{top} == -1$ then
 Output "Stack Underflow"
 Exit
2. Else
 $\text{item} = \text{stack}[\text{top}]$
 $\text{top} = \text{top} - 1$
3. End If
4. Return item

Algorithm for PEEK (View top element)

1. If $\text{top} == -1$ then
 Output "Stack is Empty"
 Exit
2. Else
 $\text{item} = \text{stack}[\text{top}]$
3. End If
4. Return item

Algorithm for isEmpty (Check if stack is empty)

1. If $\text{top} == -1$ then Return TRUE
2. Else Return FALSE

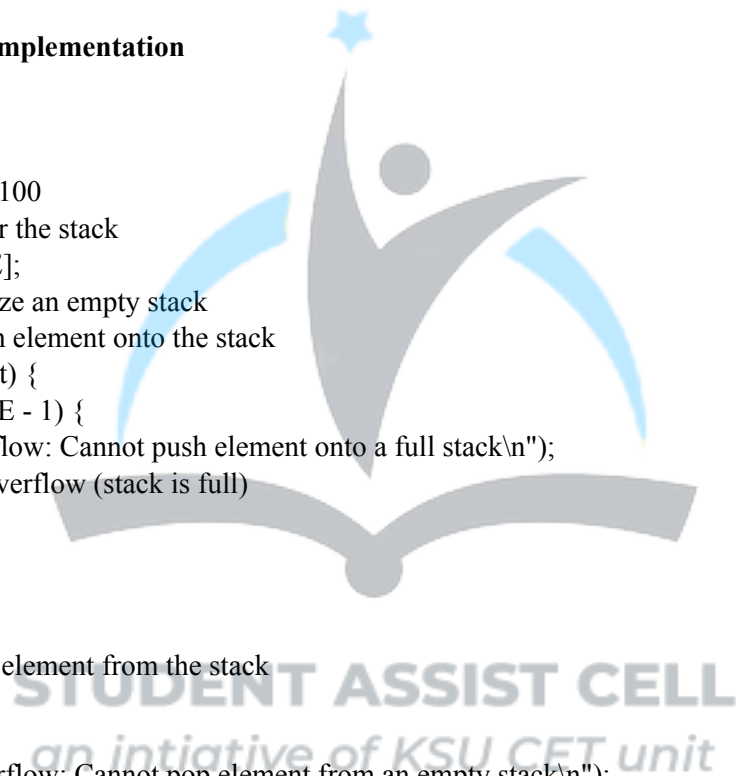


Time Complexity of Stack Operations

Operation	Time Complexity	Reason
Push (Insert)	O(1)	Add element on top, only pointer update needed
Pop (Remove)	O(1)	Remove element from top, only pointer update needed
Peek / Top	O(1)	Access the top element directly
isEmpty	O(1)	Check if the top pointer is -1 (array) or NULL (linked list)

C- Program Stack Implementation

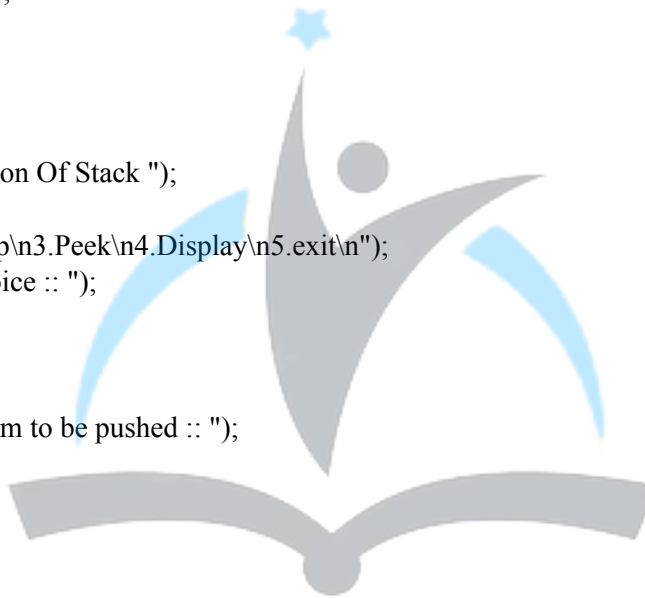
```
#include <stdio.h>
#include <stdbool.h>
#include <stdlib.h>
#define MAX_SIZE 100
// Global variables for the stack
int stack[MAX_SIZE];
int top = -1; // Initialize an empty stack
// Function to push an element onto the stack
void Push(int element) {
    if (top == MAX_SIZE - 1) {
        printf("Stack overflow: Cannot push element onto a full stack\n");
        return; // Handle overflow (stack is full)
    }
    top++;
    stack[top] = element;
}
// Function to pop an element from the stack
int Pop() {
    if (top == -1) {
        printf("Stack underflow: Cannot pop element from an empty stack\n");
        return -1; // Handle underflow (stack is empty)
    }
    int element = stack[top];
    top--;
    return element;
}
// Function to peek at the top element of the stack
int Peek() {
    if (top == -1) {
        printf("Stack is empty Underflow!\n");
        return -1; // Handle underflow (stack is empty)
    }
    return stack[top];
}
```



```

// Function to check if the stack is empty
bool IsEmpty() {
return (top == -1);
}
void Display()
{ int i;
if(IsEmpty()) printf("Stack is empty Under flow!\n");
else
{
printf("Stack contents are\n");
for(i=top;i>=0;i--)
printf("%d\n",stack[i]);
}
}
int main() {
int item, choice;
printf("\t\t\tImplementation Of Stack ");
do {
printf("\n\n1.Push\n2.Pop\n3.Peek\n4.Display\n5.exit\n");
printf("\nEnter Your Choice :: ");
scanf("%d", &choice);
switch (choice) {
case 1:
printf("\nEnter The item to be pushed :: ");
scanf("%d", &item);
Push(item);
break;
case 2:
if (IsEmpty())
printf("\nEmpty stack!Underflow !!");
else {
item = Pop();
printf("\nThe popped element is %d", item);
}
break;
case 3:
if (IsEmpty())
printf("\nEmpty stack!Underflow !!");
else {
item = Peek();
printf("\nThe Top element is %d", item);
}
break;
case 4:
Display();
break;
case 5:

```



STUDENT ASSIST CELL
an initiative of KSU CET unit

```

    exit(0);
}
} while (1);
return 0;
}

```

Complexity Analysis:

Push, Pop, Peek, and IsEmpty operations in a stack typically have a time complexity of **O(1)** when implemented using an array or linked list.

Use Cases:(Applications)

- **Function Call Stack:** Stacks are crucial in programming languages for managing function calls and maintaining the order of execution.
- **Expression Evaluation:** Stacks are used to evaluate mathematical expressions, such as converting infix expressions to postfix (RPN) or solving equations.
- **Undo/Redo Functionality:** Stacks are used to implement undo and redo functionality in applications like text editors.
- **Backtracking Algorithms:** Stacks help in backtracking algorithms like depth-first search (DFS).

MULTIPLE STACKS

- Multiple stacks mean managing two or more stacks inside a single array.
- Purpose: To save space and share memory better instead of using separate arrays for each stack.

There are two ways to implement multiple stacks.

Type	Explanation
1. Fixed Division	Divide array into fixed parts for each stack. Each stack gets its own block. Simple but can waste memory.
2. Dynamic Division	Allow stacks to grow toward each other. More efficient and avoids memory wastage.

Fixed Division (Simple Method)

Suppose array size = 10:

- Stack 1 → indexes 0 to 4
- Stack 2 → indexes 5 to 9

Each stack behaves independently within its section.

Limitations: One stack may overflow even if other has free space.

Dynamic Sharing (Best Method)

- Stack 1 grows from left to right.
- Stack 2 grows from right to left.
- Overflow occurs only if $\text{top1} + 1 == \text{top2}$.

Algorithm (Fixed Partition)

For each stack i :

- $\text{start}[i] \rightarrow$ starting index of stack i
- $\text{end}[i] \rightarrow$ ending index of stack i
- $\text{top}[i] \rightarrow$ initially $\text{start}[i] - 1$

PUSH Operation:

- Check if $\text{top}[i] < \text{end}[i]$
- If yes, increment $\text{top}[i]$ and insert element.

POP Operation:

- Check if $\text{top}[i] \geq \text{start}[i]$
- If yes, pop element and decrement $\text{top}[i]$.

Drawbacks of Fixed Partitioning

- Suppose Stack 1 fills up and Stack 2 is empty: Stack 1 can't use free space of Stack 2.
- So not memory efficient if number of stacks is large or usage is uneven.

FLEXIBLE PARTITIONING (Advanced Method)

- Dynamic allocation inside array.
- Use 3 arrays:
 - $\text{arr}[] \rightarrow$ Actual data
 - $\text{top}[] \rightarrow$ Top index for each stack
 - $\text{next}[] \rightarrow$ Helps in linking free spaces
- Also maintain a free pointer which points to the next free index.

This method is called k Stacks in a Single Array dynamically.

Advanced but very efficient.

HOW FLEXIBLE PARTITIONING WORKS?

Suppose you have:

- $\text{arr}[10]$
- $\text{top}[k]$
- $\text{next}[10]$
- $\text{free initially} = 0$

At every insertion:

- Use free index
- Update free using $\text{next}[]$

- Adjust `next []` to point to previous top of the stack
- Update `top []`

C Program: Multiple Stacks(3 Stacks) in One Array (Fixed Partitioning)

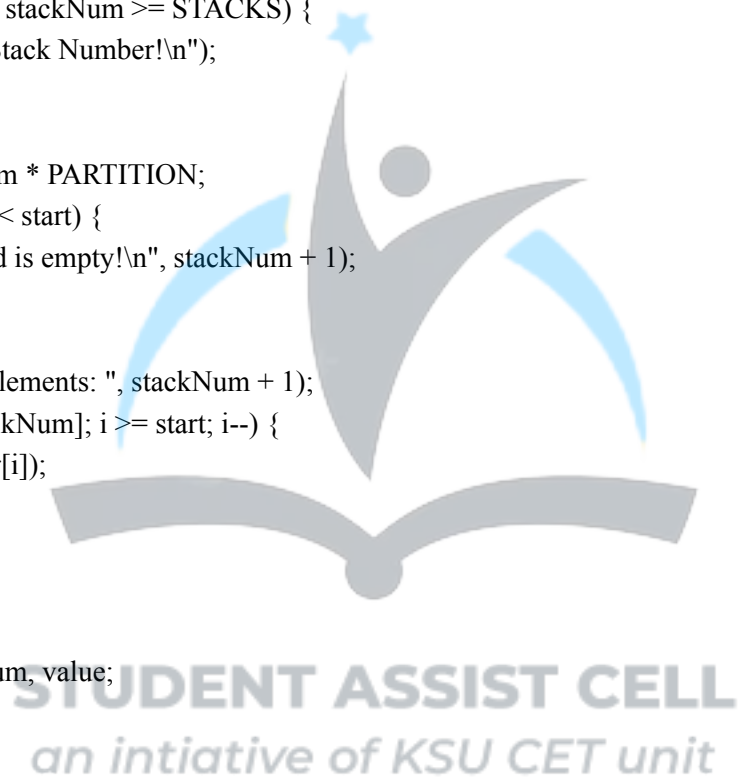
```
#include <stdio.h>
#include <stdlib.h>
#define SIZE 30 // Total size of array
#define STACKS 3 // Number of stacks
#define PARTITION (SIZE / STACKS) // Size of each stack partition
int arr[SIZE];
int top[STACKS]; // Array to store tops of stacks
// Initialize tops
void initializeTops() {
    for (int i = 0; i < STACKS; i++) {
        top[i] = i * PARTITION - 1;
    }
}
// Push into a particular stack
void push(int stackNum, int value) {
    if (stackNum < 0 || stackNum >= STACKS) {
        printf("Invalid Stack Number!\n");
        return;
    }
    // Check for overflow
    if (top[stackNum] >= ((stackNum + 1) * PARTITION) - 1) {
        printf("Stack %d Overflow!\n", stackNum + 1);
    } else {
        top[stackNum]++;
        arr[top[stackNum]] = value;
        printf("Pushed %d into Stack %d\n", value, stackNum + 1);
    }
}
// Pop from a particular stack
int pop(int stackNum) {
    if (stackNum < 0 || stackNum >= STACKS) {
        printf("Invalid Stack Number!\n");
        return -1;
    }
    int start = stackNum * PARTITION;
    // Check for underflow
    if (top[stackNum] < start) {
```

```

    printf("Stack %d Underflow!\n", stackNum + 1);
    return -1;
} else {
    int value = arr[top[stackNum]];
    top[stackNum]--;
    return value;
}
}
// Display a particular stack
void display(int stackNum) {
    if (stackNum < 0 || stackNum >= STACKS) {
        printf("Invalid Stack Number!\n");
        return;
    }
    int start = stackNum * PARTITION;
    if (top[stackNum] < start) {
        printf("Stack %d is empty!\n", stackNum + 1);
        return;
    }
    printf("Stack %d elements: ", stackNum + 1);
    for (int i = top[stackNum]; i >= start; i--) {
        printf("%d ", arr[i]);
    }
    printf("\n");
}

int main() {
    int choice, stackNum, value;
    initializeTops();
    while (1) {
        printf("\n--- Menu ---\n");
        printf("1. Push\n");
        printf("2. Pop\n");
        printf("3. Display\n");
        printf("4. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);
        switch (choice) {
            case 1:
                printf("Enter stack number (1-%d): ", STACKS);
                scanf("%d", &stackNum);
                printf("Enter value to push: ");

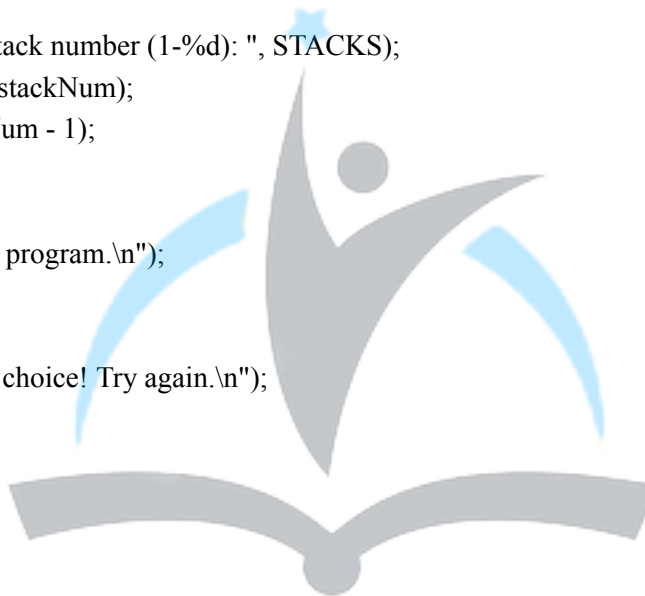
```



```

scanf("%d", &value);
push(stackNum - 1, value); // Adjust index
break;
case 2:
printf("Enter stack number (1-%d): ", STACKS);
scanf("%d", &stackNum);
value = pop(stackNum - 1);
if (value != -1)
printf("Popped value: %d\n", value);
break;
case 3:
printf("Enter stack number (1-%d): ", STACKS);
scanf("%d", &stackNum);
display(stackNum - 1);
break;
case 4:
printf("Exiting program.\n");
exit(0);
default:
printf("Invalid choice! Try again.\n");
}
}
return 0;
}

```



QUEUE USING ARRAY

A Queue is a linear data structure that follows the First In First Out (FIFO) principle. First In First Out (FIFO) means: The element added first will be removed first. Think of a queue in a bank or queue at a ticket counter: The first person to enter the queue is the first to get served.

Basic Queue Operations:

Operation	Meaning
enqueue (x)	Add element x to the rear (end) of the queue.
dequeue ()	Remove the front element from the queue.
peek ()	View the element at the front without removing it.
isEmpty ()	Check if the queue is empty.

Important Queue Terminologies

Term	Meaning
Front	Points to the first element in the queue.
Rear	Points to the last element in the queue.
Size	Total number of elements currently in the queue.

Algorithms for Basic Queue Operations

We assume:

- `queue[MAX]` is an array to hold queue elements.
- `front` and `rear` are integers.(-1 initially)
- `MAX` is the maximum size.

Algorithm for Enqueue (Insert element)

Algorithm ENQUEUE(item)

1. If `rear == MAX - 1` then
 Output "Queue Overflow"
 Exit
2. Else
 `rear = rear + 1`
 `queue[rear] = item`
4. End If
5. Return

Algorithm for Dequeue (Remove element)

Algorithm DEQUEUE()

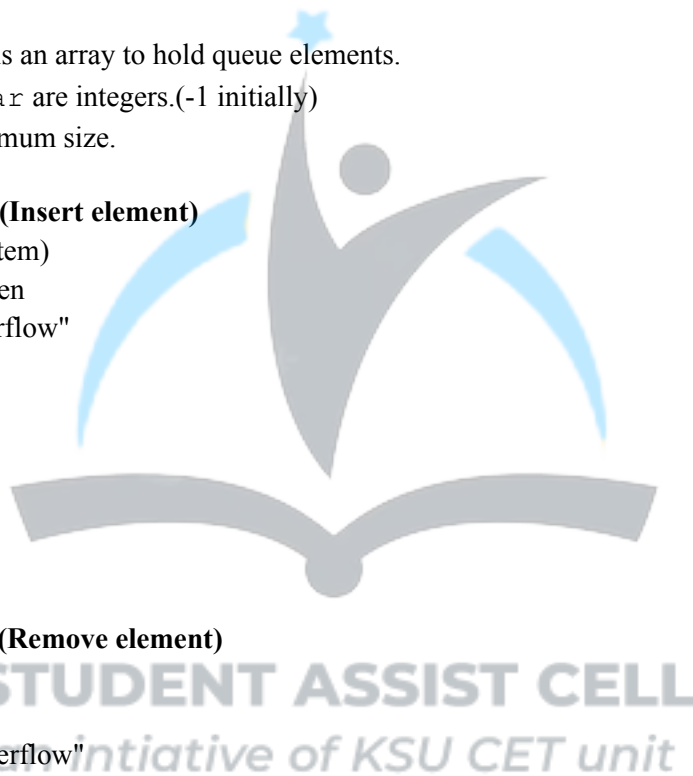
1. If `front == rear` then
 Output "Queue Underflow"
 Exit
2. Else
 `front = front + 1`
 `item = queue[front]`
3. End If
4. Return item

Note: if `front==rear` we can also reset queue to utilize space

Algorithm for Peek (View Front Element)

Algorithm PEEK()

1. If `front == rear` then
 Output "Queue is Empty"
 Exit



2. Else
 item = queue[front+1]
3. End If
4. Return item

Algorithm for isEmpty (Check if queue is empty)

Algorithm isEmpty()

1. If front == rear then
 Return TRUE
2. Else
 Return FALSE

Important Points

- **Queue Overflow** occurs when trying to add to a full queue.
- **Queue Underflow** occurs when trying to remove from an empty queue.
- In **linear queue**, after a lot of enqueues and dequeues, space at the front is wasted.
→ To solve this, we can use a **circular queue**

		Queue									
empty	index	0	1	2	3	4	5	6	7	8	9
	element										
	front=-1										
	rear=-1										
enqueued		0	1	2	3	4	5	6	7	8	9
10,20,30,40		10	20	30	40						
	front=-1										
					rear=3						
dequeued		0	1	2	3	4	5	6	7	8	9
10,20				30	40						
			front=1								
					rear=3						
enqueued		0	1	2	3	4	5	6	7	8	9
50,55,60,65,70,75				30	40	50	55	60	65	70	75
queue full			front=1								
											rear=9

Time Complexity of Queue Operations

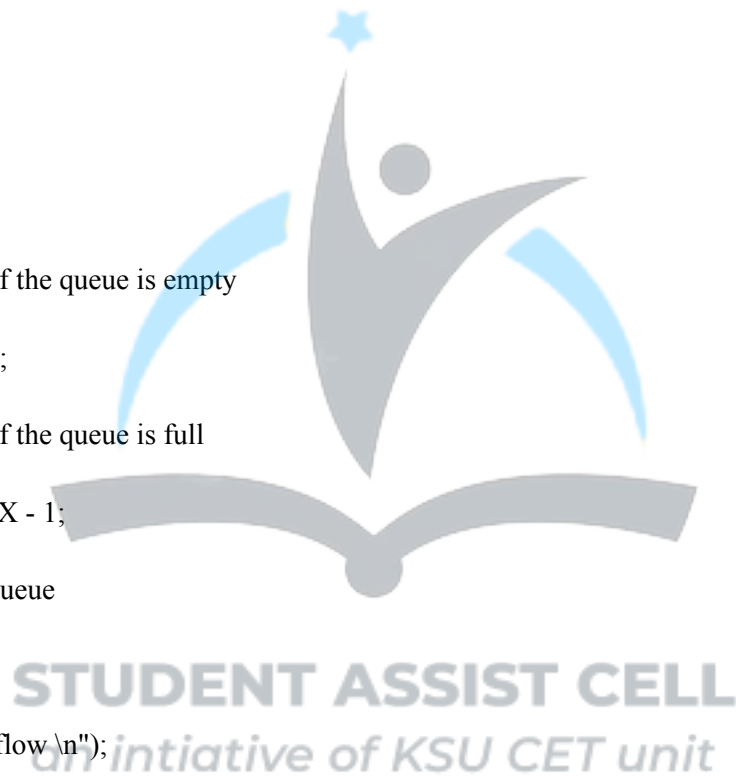
Operation	Description	Time Complexity
Enqueue (Insert)	Add an element at the rear of the queue	O(1)
Dequeue (Remove)	Remove an element from the front of the queue	O(1)
Front / Peek	Get the front element without removing it	O(1)
isEmpty	Check if the queue is empty	O(1)
isFull (for fixed-size array queues)	Check if the queue is full	O(1)

5 Important Applications of Queues

1. **Process Scheduling** — Managing processes waiting for CPU time in operating systems.
2. **Printer Queue Management** — Handling multiple print jobs in order.
3. **Handling Requests in Web Servers** — Managing incoming client requests.
4. **Breadth-First Search (BFS)** — Exploring graphs level by level.
5. **Network Packet Scheduling** — Queuing data packets in routers before transmission.

C Program - queue using arrays

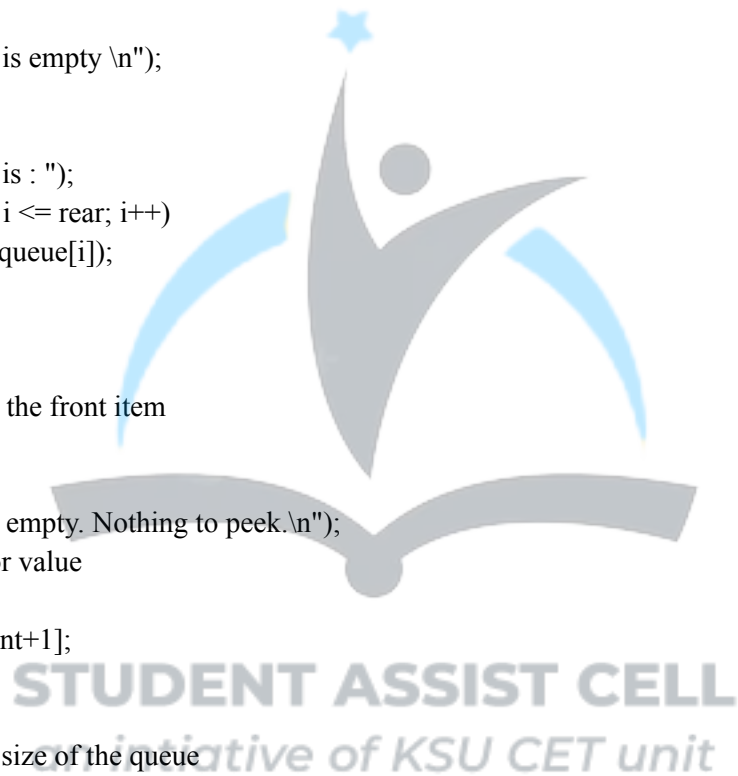
```
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>
#define MAX 10
// initializing queue
int queue[MAX];
int rear = - 1;
int front = -1;
// Function to check if the queue is empty
bool isEmpty() {
    return rear==front ;
}
// Function to check if the queue is full
bool isFull() {
    return rear == MAX - 1;
}
void enqueue() // enqueue
{
    int item;
    if (isFull())
        printf("Queue Overflow \n");
    else
    {
        printf("\nInset the element in queue : ");
        scanf("%d", &item);
        rear = rear + 1;
        queue[rear] = item;
    }
} /*End of insert()*/
void dequeue() //dequeue
{
    if (isEmpty() )
    {
        printf("\nQueue Underflow \n");
    }
}
```



```

return ;
}
else
{
front = front + 1;
printf("\nElement deleted from queue is : %d\n", queue[front]);
}
} /*End of delete() ; if front==rear we also reset queue to utilize space*/
void display()// print queue
{
int i;
if (isEmpty() )
printf("\nQueue is empty \n");
else
{
printf("\nQueue is : ");
for (i = front+1; i <= rear; i++)
printf("%d ", queue[i]);
printf("\n");
}
}
// Function to peek at the front item
int peek() {
if (isEmpty()) {
printf("Queue is empty. Nothing to peek.\n");
return -1; // Error value
} else {
return queue[front+1];
}
}
// Function to get the size of the queue
int size() {
if (isEmpty()) {
return 0;
} else {
return rear - (front + 1)+1;
}
}
int main()
{
int choice;
while (1)
{
printf("\n1.Insert element to queue \n");

```



```

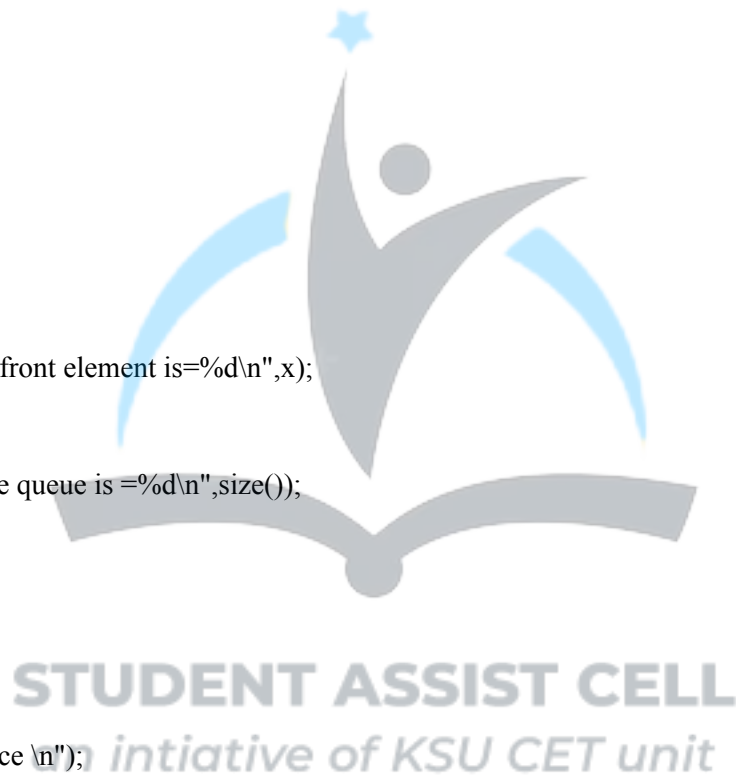
printf("2.Delete element from queue \n");
printf("3.Display all elements of queue \n");
printf("4.Peek \n");
printf("5.Size of the queue\n");
printf("6.Quit \n");
printf("\nEnter your choice : ");
scanf("%d", &choice);
switch (choice)
{
case 1:
    enqueue();
    break;
case 2:
    dequeue();
    break;
case 3:
    display();
    break;
case 4:
    int x=peek();
    if(x!=-1) printf("\nfront element is=%d\n",x);
    break;
case 5:
    printf("\nsize of the queue is =%d\n",size());
    break;
case 6:
    exit(1);

default:
printf("\nWrong choice \n");

} /*End of switch*/

} /*End of while*/
return 0;
} /*End of main()*/

```



CIRCULAR QUEUE USING ARRAY

A Circular Queue is a special form of a queue where: It behaves in a circular fashion. In a normal (linear) queue, once the rear reaches the last index of the array, we cannot insert even if there are empty spaces at the beginning. Circular Queue solves this by wrapping around.

Circular Queue Operations

Operation	Meaning
enqueue(x)	Insert an element into the queue.
dequeue()	Remove an element from the queue.
peek()	Get the front element without removing it.
isEmpty()	Check if the queue is empty.
isFull()	Check if the queue is full.

Circular Queue Algorithms

Assume:

- Queue array: queue[MAX]
- Initially: front = 0, rear = 0
- A slot is wasted to distinguish full and empty.

Algorithm for isEmpty()

Algorithm isEmpty(front, rear)

1. If front == rear
Return TRUE
2. Else
Return FALSE

Meaning:

If front == rear, queue is empty.

Algorithm for isFull()

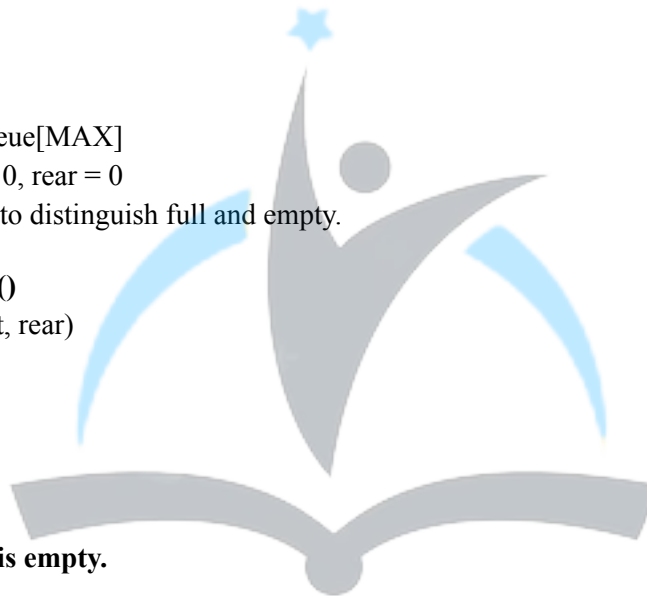
1. If (rear + 1) mod MAX == front
Return TRUE
2. Else
Return FALSE

Meaning:

If the next position of rear is front, the queue is full.

Algorithm for Enqueue (Insert an element)

1. If isFull(front, rear, MAX) then
Output "Queue Overflow"
Exit
2. Else
rear = (rear + 1) mod MAX
queue[rear] = item
3. End If

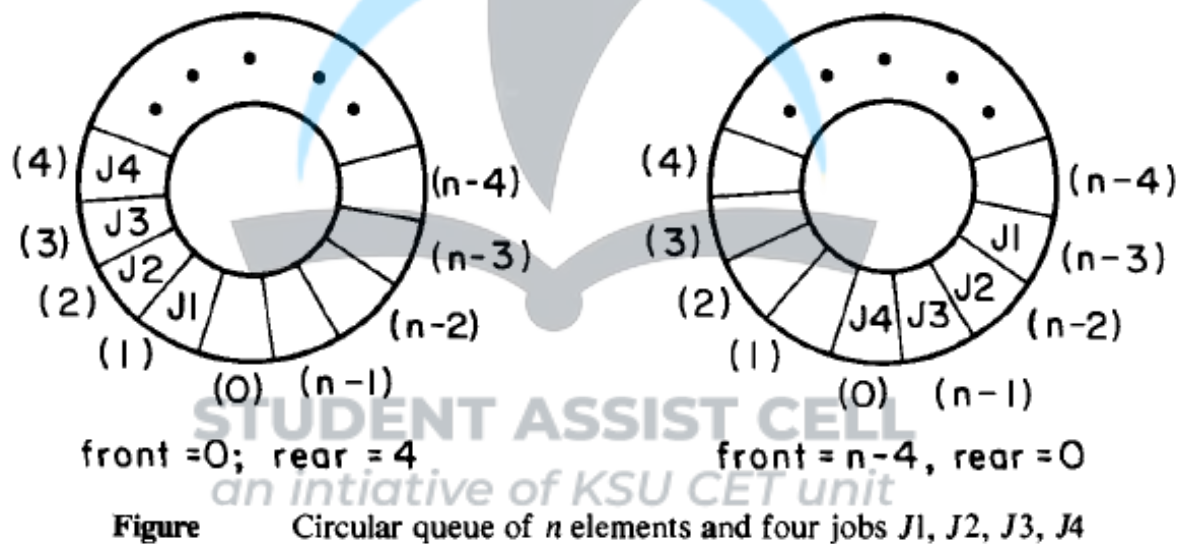


Algorithm for Dequeue (Remove an element)

1. If isEmpty(front, rear) then
 Output "Queue Underflow"
 Exit
2. Else
 $\text{front} = (\text{front} + 1) \bmod \text{MAX}$
 $\text{item} = \text{queue}[\text{front}]$
3. End If
4. Return item

Algorithm for Peek (View Front Element)

1. If isEmpty(front, rear) then
 Output "Queue is Empty"
 Exit
2. Else
 $\text{temp} = (\text{front} + 1) \bmod \text{MAX}$
 Return $\text{queue}[\text{temp}]$



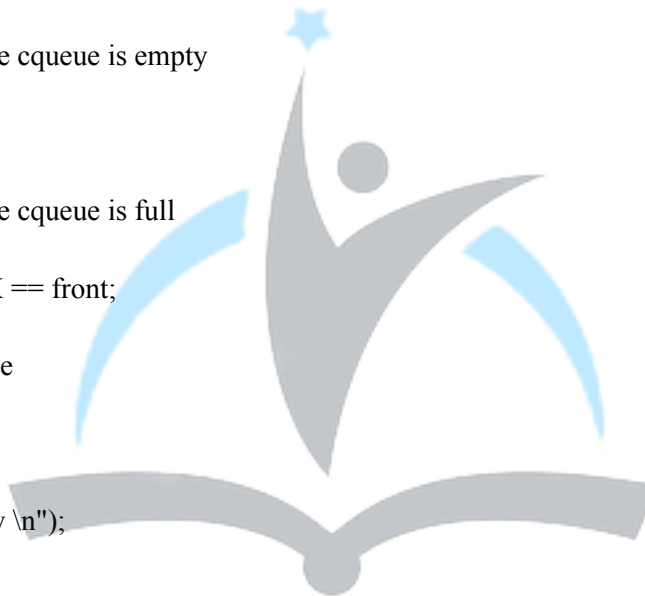
Time Complexity of Circular Queue Operations

Operation	Time Complexity	Reason
Enqueue (Insert)	$O(1)$	Insert element at rear, just update rear index (with wrap around)
Dequeue (Remove)	$O(1)$	Remove element from front, just update front index (with wrap around)
Front (Peek)	$O(1)$	Access the element at front index
Rear (Peek)	$O(1)$	Access the element at rear index

isEmpty	O(1)	Compare front and rear indices or check size
isFull	O(1)	Check if queue is full by comparing front and rear

C Program - Circular Queue using array

```
#include <stdio.h>
#include<stdlib.h>
#include <stdbool.h>
#define MAX 10
// initilazing queue
int cqueue[MAX];
int rear = 0;
int front = 0;
// Function to check if the cqueue is empty
bool isEmpty() {
    return front==rear ;
}
// Function to check if the cqueue is full
bool isFull() {
    return (rear+1)%MAX == front;
}
void enqueue() // enqueue
{
    int item;
    if (isFull())
        printf("Queue Overflow \n");
    else
    {
        printf("\nInset the element in queue : ");
        scanf("%d", &item);
        rear = (rear + 1)%MAX;
        cqueue[rear] = item;
    }
} /*End of insert()*/
void dequeue() //dequeue
{
    if(isEmpty())
    {
        printf("\nQueue Underflow \n");
        return ;
    }
    else
    {
        front = (front + 1) %MAX;
        printf("\nElement deleted from queue is : %d\n", cqueue[front]);
    }
}
```



STUDENT ASSIST CELL
an initiative of KSU CET unit

```

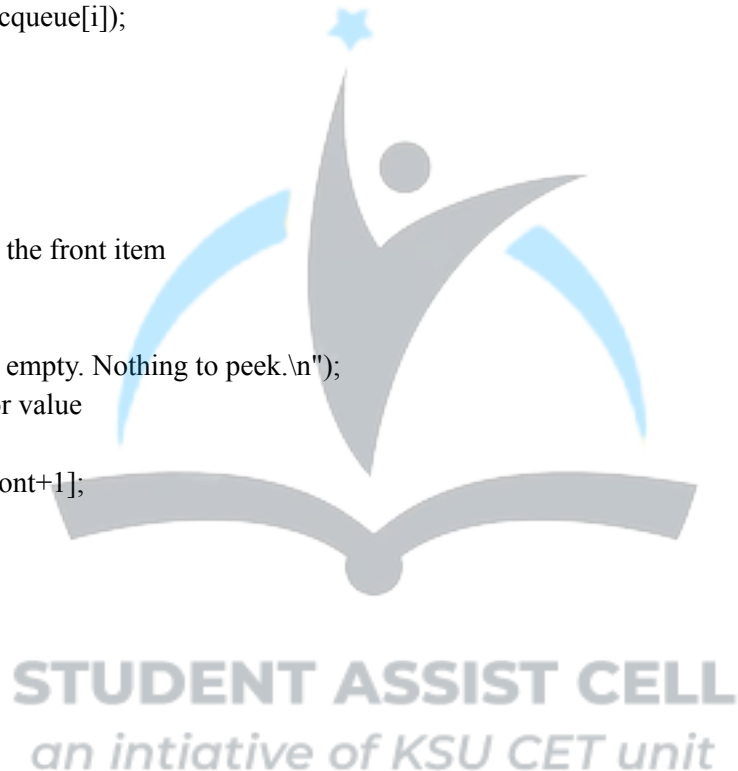
    }
} /*End of delete() */
void display()// print queue
{
int i;
if (isEmpty() )
    printf("\nQueue is empty \n");
else
    { printf("\nQueue is : ");

        for (i = (front+1)%MAX; i != (rear+1)%MAX; i=(i+1)%MAX)
            printf("%d ", cqueue[i]);

        printf("\n");

    }
}
// Function to peek at the front item
int peek() {
    if (isEmpty()) {
        printf("Queue is empty. Nothing to peek.\n");
        return -1; // Error value
    } else {
        return cqueue[front+1];
    }
}
int main()
{
int choice;
while (1)
{
printf("\n1.Insert element to queue \n");
printf("2.Delete element from queue \n");
printf("3.Display all elements of queue \n");
printf("4.Peek \n");
printf("5.Quit \n");
printf("\nEnter your choice : ");
scanf("%d", &choice);
switch (choice)
{
case 1:
    enqueue();
    break;
case 2:

```



```

    dequeue();
    break;
case 3:
    display();
    break;
case 4:
    int x=peek();
    if(x!=-1) printf("\nfront element is=%d\n",x);
    break;
case 5:
    exit(1);

default:

printf("\nWrong choice \n");

} /*End of switch*/

} /*End of while*/
return 0;
} /*End of main()*/

```

DOUBLE ENDED QUEUE USING ARRAY

A Deque stands for Double Ended Queue.

It is a linear data structure in which insertion and deletion operations can be performed from both front and rear ends. Unlike a simple queue (where insert at rear and delete from front only), in Deque:

- Insert front
- Insert rear
- Delete front
- Delete rear

STUDENT ASSIST CELL
an initiative of KSU CET unit

Types of Deques

There are two common types:

1. Input Restricted Deque
 - Insertion at only one end (rear).
 - Deletion from both ends.
2. Output Restricted Deque
 - Deletion at only one end (front).
 - Insertion from both ends.

Basic Operations in a General Deque:

Operation	Description
Insert Front	Insert an element at the front end.
Insert Rear	Insert an element at the rear end.
Delete Front	Delete an element from the front end.
Delete Rear	Delete an element from the rear end.
Peek Front	Get the front element without removing it.
Peek Rear	Get the rear element without removing it.
isFull()	Check if the deque is full.
isEmpty()	Check if the deque is empty.

Algorithms (assuming circular array with front = -1, rear = -1 initially)

Algorithm Initialize Deque

```
front = -1  
rear = -1
```

Algorithm for isEmpty()

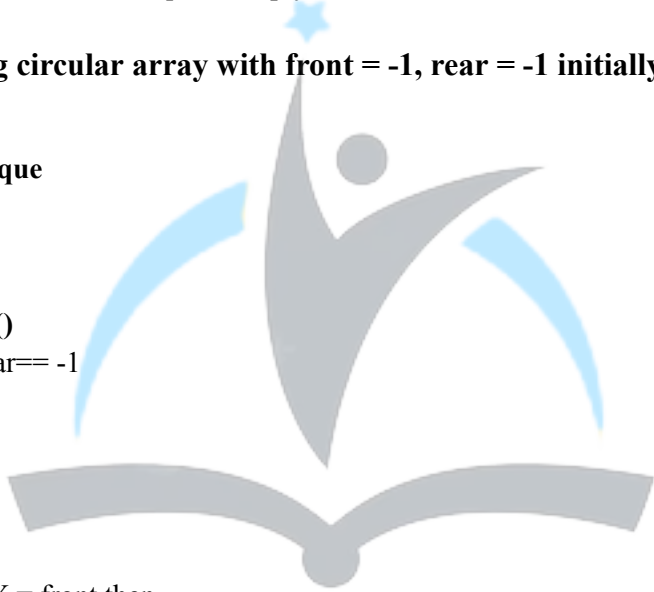
1. If front == -1 AND rear == -1
Return TRUE
2. Else
Return FALSE

Algorithm for isFull()

1. If $(\text{rear} + 1) \bmod \text{MAX} = \text{front}$ then
Return TRUE
2. Else
Return FALSE

Algorithm for Insert Front

1. If Deque is Full
Output "Deque Overflow"
Exit
2. If Deque is Empty
Set front $\leftarrow 0$
Set rear $\leftarrow 0$
3. Else
Set front $\leftarrow (\text{front} - 1 + \text{MAX}) \bmod \text{MAX}$
4. Set deque[front] $\leftarrow$ item



STUDENT ASSIST CELL
an initiative of KSU CET unit

Algorithm for Insert Rear

1. If Deque is Full
 Output "Deque Overflow"
 Exit
2. If Deque is Empty
 Set front $\leftarrow 0$
 Set rear $\leftarrow 0$
3. Else
 Set rear $\leftarrow (\text{rear} + 1) \bmod \text{MAX}$
4. Set deque[rear] $\leftarrow \text{item}$

Algorithm for Delete Front

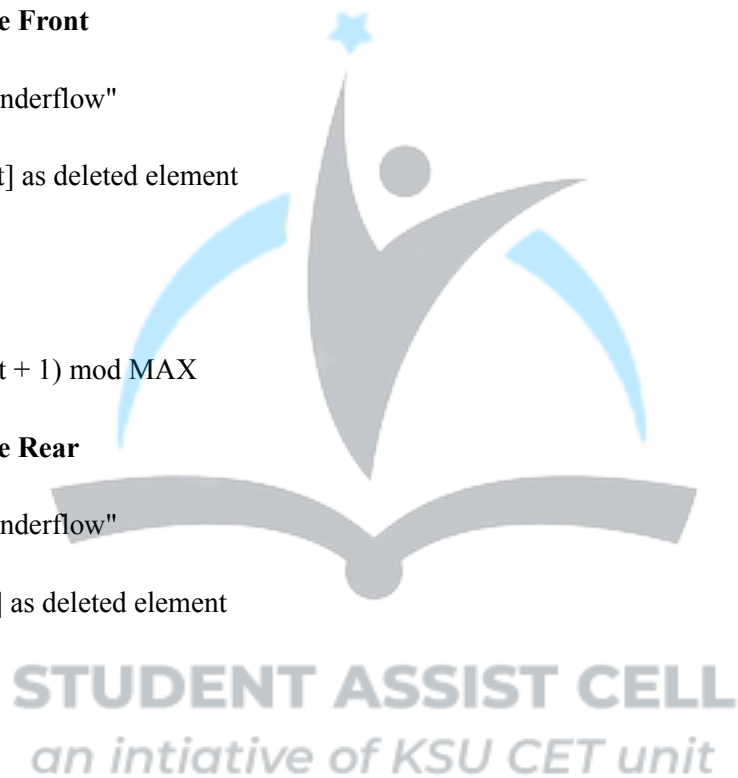
1. If Deque is Empty
 Output "Deque Underflow"
 Exit
2. Output deque[front] as deleted element
3. If front = rear
 Set front $\leftarrow -1$
 Set rear $\leftarrow -1$
4. Else
 Set front $\leftarrow (\text{front} + 1) \bmod \text{MAX}$

Algorithm for Delete Rear

1. If Deque is Empty
 Output "Deque Underflow"
 Exit
2. Output deque[rear] as deleted element
3. If front = rear
 Set front $\leftarrow -1$
 Set rear $\leftarrow -1$
4. Else
 Set rear $\leftarrow (\text{rear} - 1 + \text{MAX}) \bmod \text{MAX}$

Display Deque Elements

1. If Deque is Empty
 Output "Deque is empty"
 Exit
2. Set i $\leftarrow \text{front}$
3. Repeat
 Output deque[i]
 Set i $\leftarrow (i + 1) \bmod \text{MAX}$
Until i = (rear + 1) mod MAX



Peek Front Element

1. If front = -1 then
 Output "Deque is empty"
 Exit
2. Else
 Output deque[front] as the front element

Peek Rear Element

1. If rear = -1 then
 Output "Deque is empty"
 Exit
2. Else
 Output deque[rear] as the rear element

Note:

- Wrapping is handled using % MAX (modulus operator).
- In Deque, we can move front and rear both ways (increment/decrement).
- You can design Deque using Arrays (fixed size) or Linked Lists (dynamic size).

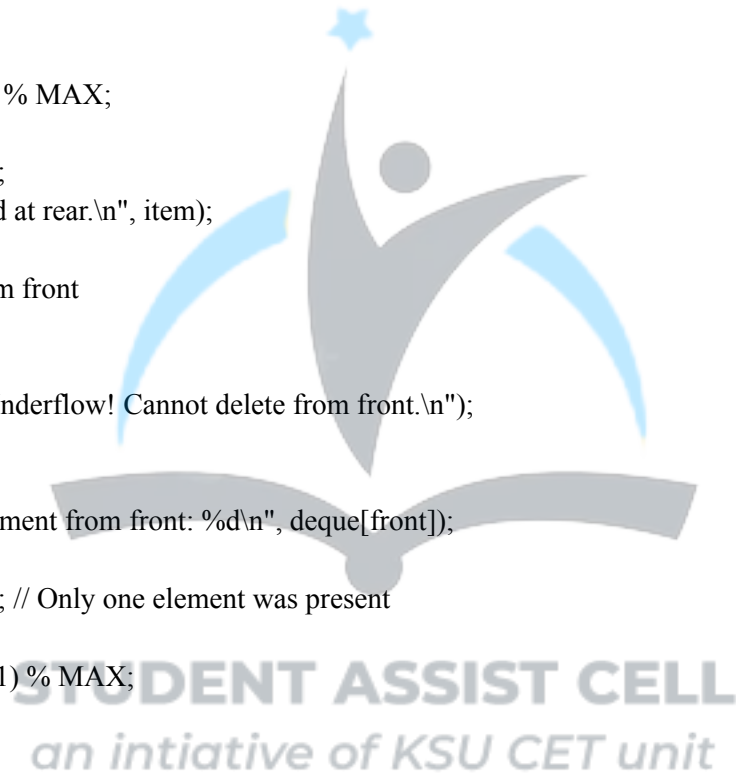
C Program Double Ended Queue using array

```
#include <stdio.h>
#include <stdlib.h>
#define MAX 5
int deque[MAX];
int front = -1, rear = -1;
// Function to check if deque is full
int isFull() {
    return ((rear + 1) % MAX == front);
}
// Function to check if deque is empty
int isEmpty() {
    return (front == -1);
}
// Insert element at front
void insertFront(int item) {
    if (isFull()) {
        printf("Deque Overflow! Cannot insert at front.\n");
        return;
    }
    if (isEmpty()) {
        front = rear = 0;
    } else {
        front = (front - 1 + MAX) % MAX;
```

```

    }
    deque[front] = item;
    printf("Inserted %d at front.\n", item);
}
// Insert element at rear
void insertRear(int item) {
    if (isFull()) {
        printf("Deque Overflow! Cannot insert at rear.\n");
        return;
    }
    if (isEmpty()) {
        front = rear = 0;
    } else {
        rear = (rear + 1) % MAX;
    }
    deque[rear] = item;
    printf("Inserted %d at rear.\n", item);
}
// Delete element from front
void deleteFront() {
    if (isEmpty()) {
        printf("Deque Underflow! Cannot delete from front.\n");
        return;
    }
    printf("Deleted element from front: %d\n", deque[front]);
    if (front == rear) {
        front = rear = -1; // Only one element was present
    } else {
        front = (front + 1) % MAX;
    }
}
// Delete element from rear
void deleteRear() {
    if (isEmpty()) {
        printf("Deque Underflow! Cannot delete from rear.\n");
        return;
    }
    printf("Deleted element from rear: %d\n", deque[rear]);
    if (front == rear) {
        front = rear = -1; // Only one element was present
    } else {
        rear = (rear - 1 + MAX) % MAX;
    }
}
}

```



```

// Display deque elements
void display() {
    int i;
    if (isEmpty()) {
        printf("Deque is empty.\n");
        return;
    }
    printf("Deque elements: ");
    i = front;
    while (1) {
        printf("%d ", deque[i]);
        if (i == rear)
            break;
        i = (i + 1) % MAX;
    }
    printf("\n");
}

```

// Main function

```

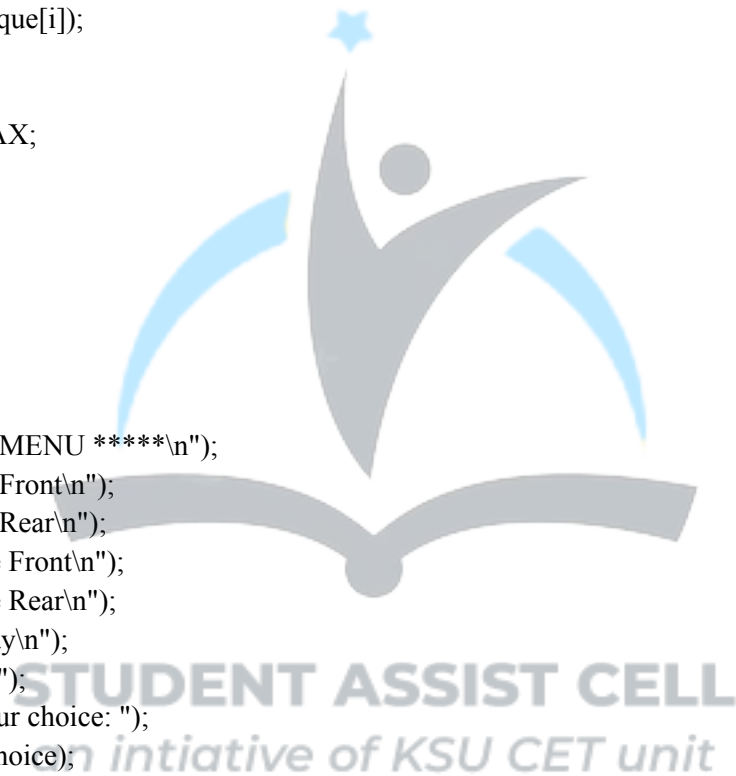
int main() {
    int choice, item;
    while (1) {
        printf("\n***** MENU *****\n");
        printf("1. Insert Front\n");
        printf("2. Insert Rear\n");
        printf("3. Delete Front\n");
        printf("4. Delete Rear\n");
        printf("5. Display\n");
        printf("6. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

```

```

switch (choice) {
    case 1:
        printf("Enter the element to insert at front: ");
        scanf("%d", &item);
        insertFront(item);
        break;
    case 2:
        printf("Enter the element to insert at rear: ");
        scanf("%d", &item);
        insertRear(item);
        break;
    case 3:

```



```

        deleteFront();
        break;
    case 4:
        deleteRear();
        break;
    case 5:
        display();
        break;
    case 6:
        printf("Exiting program.\n");
        exit(0);
    default:
        printf("Invalid choice! Please try again.\n");
    }
}
return 0;
}

```

INFIX TO POSTFIX CONVERSION AND EVALUATION

An infix expression is the way we normally write mathematical expressions: The operator is between the operands. Examples of Infix expressions:

- $A + B$
- $5 * 3$
- $5 + 3 * 2$
- $(A + B) * (C - D)$
- $(7 + 3) * (5 - 2)$

This is natural for humans because we understand precedence (multiplication first, then addition) and use parentheses.

Postfix Expression is also called Reverse Polish Notation. A postfix expression places the operator after its operands.

- No parentheses are needed.
- No precedence rules needed — order of operations is built-in.

Examples of Postfix expressions:

- $A B + \rightarrow$ (means $A + B$)
- $5 3 * \rightarrow$ (means $5 * 3$)
- $A B + C D - * \rightarrow$ (means $(A + B) * (C - D)$)
- $7 3 + 5 2 - * \rightarrow$ (means $(7 + 3) * (5 - 2)$)

Computers prefer postfix because it's very easy to evaluate using a stack.

Quick Comparison:

Feature	Infix	Postfix
Operator position	Between operands	After operands
Need for parentheses?	Yes (sometimes)	No
Operator precedence important?	Yes	No
Example	$A + B$	$A B +$
Human or Machine friendly?	Human	Machine

Examples:

Infix

$(A + B) * (C - D) / (E + F)$

$(5 + 2) * (8 - 3) / (4 + 1)$

$((A + B) * (C + D)) / ((E + F) * (G + H))$

$A + B * (C ^ D - E) ^ (F + G * H) - I$

Postfix

$A B + C D - * E F + /$

$5 2 + 8 3 - * 4 1 + /$

$A B + C D + * E F + G H + * /$

$A B C D ^ E - F G H * + ^ * + I -$

Algorithm for Infix to Postfix Conversion

We use a stack to hold operators and parentheses during the conversion.

Steps:

1. Initialize an empty stack for operators and an empty postfix expression.
2. Scan the infix expression from left to right.
3. For each symbol in the infix expression:
 - If the symbol is an operand (letter or number), add it to the postfix expression.
 - If the symbol is $($, push it onto the stack.
 - If the symbol is $)$:
 - Pop from the stack and append to the postfix expression until a $($ is encountered.
 - Pop and discard the $($.
 - If the symbol is an operator $(+, -, *, /, ^)$:
 - While the stack is not empty, and the top of the stack has higher or equal precedence than the current operator:
 - Pop from the stack and append to the postfix expression.

- Push the current operator onto the stack.
- 4. After the entire infix expression is scanned:
 - Pop all remaining operators from the stack and append them to the postfix expression

Example:

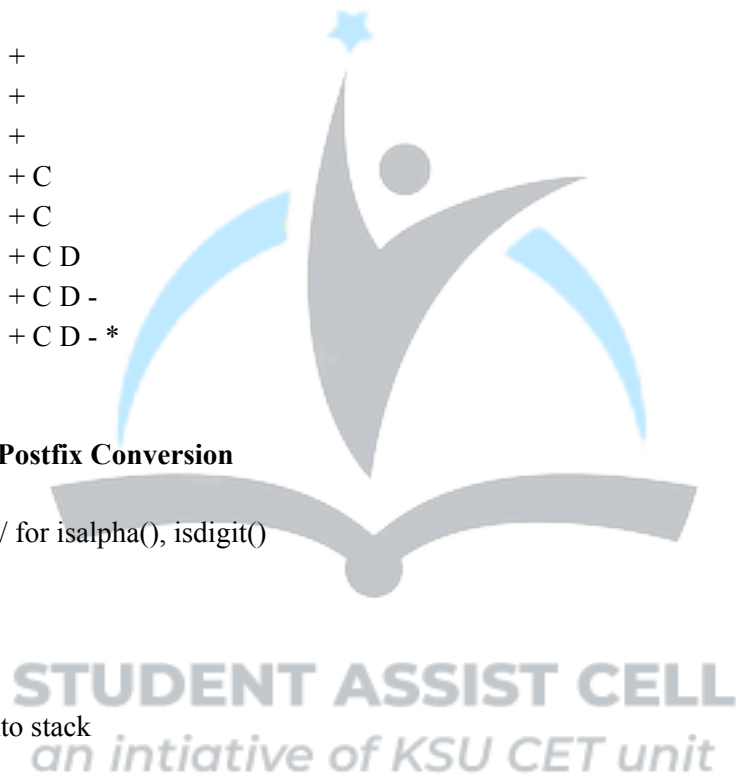
$(A + B) * (C - D)$

Input	Stack	Postfix
(	(	
A	(	A
+	(+	A
B	(+	A B
)		A B +
*	*	A B +
(	* (	A B +
C	* (	A B + C
-	* (-	A B + C
D	* (-	A B + C D
)	*	A B + C D -
(end)		A B + C D - *

C Program Infix to Postfix Conversion

```
#include <stdio.h>
#include <ctype.h> // for isalpha(), isdigit()
#include <string.h>
#define MAX 100
char stack[MAX];
int top = -1;
// Function to push into stack
void push(char c) {
    stack[++top] = c;
}
// Function to pop from stack
char pop() {
    if (top == -1)
        return -1;
    else
        return stack[top--];
}
// Function to return precedence of operators
int precedence(char c) {
    if (c == '^')
        return 3;

```



```

else if (c == '*' || c == '/')
    return 2;
else if (c == '+' || c == '-')
    return 1;
else
    return 0;
}

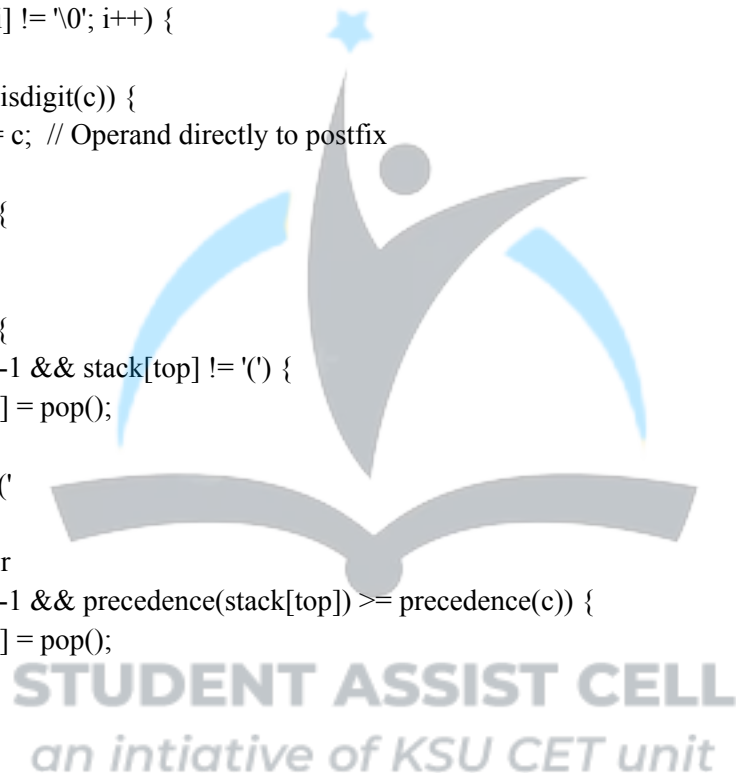
void infixToPostfix(char infix[]) {
    char postfix[MAX];
    int i, j = 0;
    char c;
    for (i = 0; infix[i] != '\0'; i++) {
        c = infix[i];
        if (isalpha(c) || isdigit(c)) {
            postfix[j++] = c; // Operand directly to postfix
        }
        else if (c == '(') {
            push(c);
        }
        else if (c == ')') {
            while (top != -1 && stack[top] != '(') {
                postfix[j++] = pop();
            }
            pop(); // pop '('
        }
        else { // Operator
            while (top != -1 && precedence(stack[top]) >= precedence(c)) {
                postfix[j++] = pop();
            }
            push(c);
        }
    }

    // Pop any remaining operators
    while (top != -1) {
        postfix[j++] = pop();
    }

    postfix[j] = '\0'; // End postfix expression
    printf("Postfix Expression: %s\n", postfix);
}

int main() {
    char infix[MAX];
    printf("Enter an Infix Expression: ");
    scanf("%s", infix);
    infixToPostfix(infix);
}

```



```

return 0;
}

```

Output

Enter an Infix Expression: $(A+B)*(C-D)/F^E$

Postfix Expression: $AB+CD-*FE^/$

Evaluate a Postfix Expression

We use a stack:

Algorithm Evaluation of Postfix Expression

1. Scan the postfix expression left to right.
2. For each symbol:
 - If it's an operand (number), push it onto the stack.
 - If it's an operator:
 - Pop the top two elements from the stack.
 - Apply the operator (second popped item operator first popped item).
 - Push the result back into the stack.
3. At the end, the stack contains only one element — the final result.

Simple Example: single digit

$7\ 8 + 3\ 2 + /$

Step-by-step:

Symbol Stack Explanation

7	7	Push 7
8	7, 8	Push 8
+	15	$7 + 8 = 15$
3	15, 3	Push 3
2	15, 3, 2	Push 2
+	15, 5	$3 + 2 = 5$
/	3	$15 / 5 = 3$

Final result: 3

Example Problems:

Infix	Postfix	Result
$(5 * (6 + 2)) - (12 / 4)$	$5\ 6\ 2 + * 12\ 4 / -$	37

$(7 + 8) / (3 + 2)$

$7\ 8 + 3\ 2 + /$

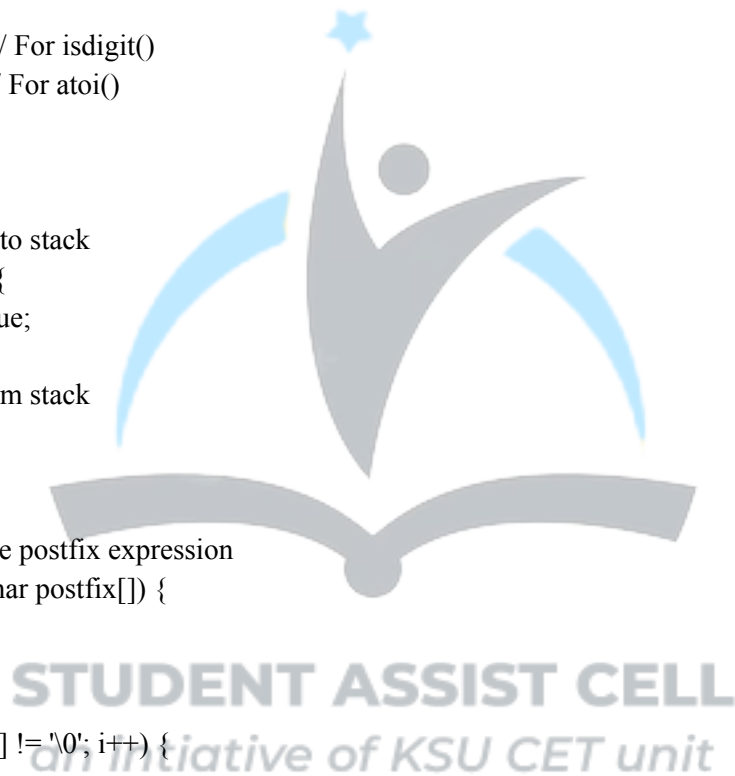
3

Note

- Always push operands.
- Always pop two operands for an operator.
- Order matters: second popped value is first operand, first popped value is second operand.
- Use a stack to keep things simple.

Implementation in C: single digit

```
#include <stdio.h>
#include <ctype.h> // For isdigit()
#include <stdlib.h> // For atoi()
#define MAX 100
int stack[MAX];
int top = -1;
// Function to push into stack
void push(int value) {
    stack[++top] = value;
}
// Function to pop from stack
int pop() {
    return stack[top--];
}
// Function to evaluate postfix expression
int evaluatePostfix(char postfix[]) {
    int i;
    char ch;
    int val;
    for (i = 0; postfix[i] != '\0'; i++) {
        ch = postfix[i];
        if (isdigit(ch)) {
            // If operand, push onto stack
            push(ch - '0'); // Convert char to int
        }
        else if (ch == ' ') {
            // Ignore spaces
            continue;
        }
        else {
            // Operator encountered
            int val2 = pop();
            int val1 = pop();
```



```

switch (ch) {
    case '+':
        push(val1 + val2);
        break;
    case '-':
        push(val1 - val2);
        break;
    case '*':
        push(val1 * val2);
        break;
    case '/':
        push(val1 / val2);
        break;
    case '^':
        {
            int result = 1;
            for (int i = 0; i < val2; i++)
                result *= val1;
            push(result);
            break;
        }
    }
}
return pop();
}

int main() {
    char postfix[MAX];
    printf("Enter a Postfix Expression (single-digit numbers and operators without spaces, like 562+*): ");
    scanf("%s", postfix);
    int result = evaluatePostfix(postfix);
    printf("Result of Postfix Evaluation: %d\n", result);
    return 0;
}

```

Output

Enter a Postfix Expression (single-digit numbers and operators without spaces, like 562+*): 562+*
 Result of Postfix Evaluation: 40